EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

LOW CODE APPLICATION • LABORATORY REPORT

Student Name	Aravindh Shankar
Roll Number	241401012
Programme / Year	B.Tech Computer Science and Business Systems — III Year
Institution	Rajalakshmi Engineering College
Course	Low Code Application

Experiment aim

To evaluate where low-code platforms accelerate development and where complexity, scale, integration, and governance requirements may demand pro-code components or a different architecture.

Application / case context

The Gym Enrollment app is a small, well-bounded workflow and is suitable for a low-code tool. To study platform limits, this report considers a hypothetical enterprise case: a multi-region hospital resource orchestration system that coordinates beds, staff, operating theatres, diagnostics, and emergency escalations. This is a case study, not a claim that the gym app already implements hospital operations.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

Student: Aravindh Shankar • Roll No. 241401012 • Rajalakshmi Engineering College

1. Case Study and Requirements

Enterprise use case: a hospital network needs to coordinate beds, staff rosters, operating theatre schedules, diagnostic equipment, patient movement, and urgent escalations across multiple sites. Data changes frequently and some decisions are time-critical.

Functional requirements include live resource status, role-specific screens, audit trails, integration with electronic health records, scheduling constraints, escalation policies, and analytics. Non-functional requirements include high availability, low latency, data privacy, resilience, and predictable behaviour under load.

Low-code platforms can rapidly build forms, administrative workflows, reports, and internal dashboards. The challenge is whether the platform can meet the strictest timing, scale, integration, and governance requirements of the full system.

Low-code suitability should be assessed against explicit requirements rather than broad claims that the platform is either easy or limited. Forms and approvals may be a strong fit, while a high-frequency optimization engine or a strict real-time service may need a different runtime. The decisive question is whether the complete workload can meet measurable requirements safely and maintainably.

A hybrid architecture can preserve the speed of low-code for user-facing workflows while isolating complex computation in a tested service. The boundary should be explicit: the low-code application collects and displays information, while a service performs specialized scheduling or optimization and returns a versioned response. This prevents complex algorithms from becoming scattered across visual workflows.

Before selecting a platform, test the riskiest requirement with realistic data. Measure latency, throughput, recovery from integration failures, and the effort needed to change business rules. Include licensing, training, migration, vendor dependence, and long-term maintenance in the cost comparison rather than comparing only initial build speed.

The case study supports a selective adoption strategy: begin with low-risk workflows, validate integration boundaries, and move higher-criticality functions only when performance, reliability, and governance have been demonstrated. Architecture decisions should be justified with evidence.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

Objectives and scope

This experiment evaluates a complex enterprise scenario and identifies where low-code is useful, where it may be constrained, and how a hybrid architecture can divide responsibilities.

2. Where Low-Code Helps

Low-code reduces repetitive UI and CRUD development. It is useful for internal forms, approval workflows, administrative portals, proof-of-concept interfaces, and reports that can be configured by domain teams.

Visual workflows help non-specialists understand process steps, and built-in authentication or connectors can reduce integration effort. A small team can validate requirements earlier than if every screen and data-entry operation were coded from scratch.

For the hospital scenario, a low-code tool might be appropriate for a non-critical staffing request form, equipment maintenance request, or administrative dashboard, provided access and audit requirements are satisfied.

Low-code suitability should be assessed against explicit requirements rather than broad claims that the platform is either easy or limited. Forms and approvals may be a strong fit, while a high-frequency optimization engine or a strict real-time service may need a different runtime. The decisive question is whether the complete workload can meet measurable requirements safely and maintainably.

A hybrid architecture can preserve the speed of low-code for user-facing workflows while isolating complex computation in a tested service. The boundary should be explicit: the low-code application collects and displays information, while a service performs specialized scheduling or optimization and returns a versioned response. This prevents complex algorithms from becoming scattered across visual workflows.

Before selecting a platform, test the riskiest requirement with realistic data. Measure latency, throughput, recovery from integration failures, and the effort needed to change business rules. Include licensing, training, migration, vendor dependence, and long-term maintenance in the cost comparison rather than comparing only initial build speed.

The case study supports a selective adoption strategy: begin with low-risk workflows, validate integration boundaries, and move higher-criticality functions only when performance, reliability, and governance have been demonstrated. Architecture decisions should be justified with evidence.

Platform limits should be validated using a proof of concept rather than inferred from marketing language. A test should use representative concurrent users, realistic record sizes, and the same integrations planned for the final system. Record both successful response time and behaviour during failure.

Migration and vendor dependence also matter. If a workflow is implemented in platform-specific scripts, moving it elsewhere may require rewriting logic and rebuilding permissions. Keep business rules documented and separate core algorithms from UI configuration when portability is important.

Technical risk should be prioritized by impact and likelihood. A slow dashboard may be inconvenient, while an incorrect hospital resource allocation could be safety-critical. The more consequential the failure, the stronger the requirements for testing, monitoring, audit trails, and human oversight.

Low-code and pro-code should be compared on the same requirements. A visual workflow may be easier to understand, while a custom service may be easier to unit-test or profile. Neither is automatically superior; choose the implementation that makes correctness and maintenance easiest to demonstrate.

Design / configuration summary

Requirement	Low-code fit	Possible complementary approach
CRUD forms and approvals	Strong	Use Creator forms/workflows
Operational dashboards	Often strong	Creator reports/pages
Complex optimization	Variable / may be weak	Dedicated tested service
Strict real-time processing	Must be measured	Event-driven services / streaming
EHR integration	Depends on connector and standards	API gateway / integration middleware
Extensive automated testing	May require extra effort	CI tests for custom services

3. Limitations and Risk Areas

Complex optimization: bed and staff allocation may involve many constraints and trade-offs that are difficult to express and test as long visual workflows. A dedicated optimization service may be more maintainable.

Real-time and high-volume processing: strict latency targets, streaming events, and large concurrent workloads may exceed comfortable platform limits or require careful architecture. Limits must be measured, not assumed.

Integration and vendor dependence: proprietary connectors, platform-specific scripting, licensing, and deployment constraints can increase switching costs. Complex EHR integration may require standards-aware middleware and extensive validation.

Testing and version control: visual configuration can be difficult to diff, review, and test automatically compared with conventional source code. Complex business rules may become opaque when distributed across workflows and custom functions.

Low-code suitability should be assessed against explicit requirements rather than broad claims that the platform is either easy or limited. Forms and approvals may be a strong fit, while a high-frequency optimization engine or a strict real-time service may need a different runtime. The decisive question is whether the complete workload can meet measurable requirements safely and maintainably.

A hybrid architecture can preserve the speed of low-code for user-facing workflows while isolating complex computation in a tested service. The boundary should be explicit: the low-code application collects and displays information, while a service performs specialized scheduling or optimization and returns a versioned response. This prevents complex algorithms from becoming scattered across visual workflows.

Before selecting a platform, test the riskiest requirement with realistic data. Measure latency, throughput, recovery from integration failures, and the effort needed to change business rules. Include licensing, training, migration, vendor dependence, and long-term maintenance in the cost comparison rather than comparing only initial build speed.

The case study supports a selective adoption strategy: begin with low-risk workflows, validate integration boundaries, and move higher-criticality functions only when performance, reliability, and governance have been demonstrated. Architecture decisions should be justified with evidence.

Platform limits should be validated using a proof of concept rather than inferred from marketing language. A test should use representative concurrent users, realistic record sizes, and the same integrations planned for the final system. Record both successful response time and behaviour during failure.

Migration and vendor dependence also matter. If a workflow is implemented in platform-specific scripts, moving it elsewhere may require rewriting logic and rebuilding permissions. Keep business rules documented and separate core algorithms from UI configuration when portability is important.

Technical risk should be prioritized by impact and likelihood. A slow dashboard may be inconvenient, while an incorrect hospital resource allocation could be safety-critical. The more consequential the failure, the stronger the requirements for testing, monitoring, audit trails, and human oversight.

Low-code and pro-code should be compared on the same requirements. A visual workflow may be easier to understand, while a custom service may be easier to unit-test or profile. Neither is automatically superior; choose the implementation that makes correctness and maintenance easiest to demonstrate.

Procedure and configuration checklist

- List functional and non-functional requirements.
- Identify the highest-risk requirement and prototype it.
- Compare low-code, pro-code, and hybrid options.
- Evaluate reliability, cost, integration, and governance.

4. Proposed Hybrid Architecture

Use the low-code platform for user-facing forms, operational dashboards, approval steps, and non-critical administrative workflows. Keep the core scheduling/optimization engine in a tested service written in a suitable pro-code stack.

Use an API layer to validate requests, apply authorization, and communicate with external systems. Use a durable queue or event mechanism for non-immediate tasks, with retry and idempotency controls. Keep an audit log of decisions and source data.

Define clear ownership boundaries: Creator handles user interaction and simple workflow orchestration; the service layer handles complex algorithms, high-volume processing, and integrations that require stronger test automation.

Low-code suitability should be assessed against explicit requirements rather than broad claims that the platform is either easy or limited. Forms and approvals may be a strong fit, while a high-frequency optimization engine or a strict real-time service may need a different runtime. The decisive question is whether the complete workload can meet measurable requirements safely and maintainably.

A hybrid architecture can preserve the speed of low-code for user-facing workflows while isolating complex computation in a tested service. The boundary should be explicit: the low-code application collects and displays information, while a service performs specialized scheduling or optimization and returns a versioned response. This prevents complex algorithms from becoming scattered across visual workflows.

Before selecting a platform, test the riskiest requirement with realistic data. Measure latency, throughput, recovery from integration failures, and the effort needed to change business rules. Include licensing, training, migration, vendor dependence, and long-term maintenance in the cost comparison rather than comparing only initial build speed.

The case study supports a selective adoption strategy: begin with low-risk workflows, validate integration boundaries, and move higher-criticality functions only when performance, reliability, and governance have been demonstrated. Architecture decisions should be justified with evidence.

Platform limits should be validated using a proof of concept rather than inferred from marketing language. A test should use representative concurrent users, realistic record sizes, and the same integrations planned for the final system. Record both successful response time and behaviour during failure.

Migration and vendor dependence also matter. If a workflow is implemented in platform-specific scripts, moving it elsewhere may require rewriting logic and rebuilding permissions. Keep business rules documented and separate core algorithms from UI configuration when portability is important.

Technical risk should be prioritized by impact and likelihood. A slow dashboard may be inconvenient, while an incorrect hospital resource allocation could be safety-critical. The more consequential the failure, the stronger the requirements for testing, monitoring, audit trails, and human oversight.

Low-code and pro-code should be compared on the same requirements. A visual workflow may be easier to understand, while a custom service may be easier to unit-test or profile. Neither is automatically superior; choose the implementation that makes correctness and maintenance easiest to demonstrate.

Verification checklist

- Test realistic workload and failure scenarios.
- Review maintainability and platform dependence.
- Validate the chosen architecture against requirements.

5. Evaluation and Conclusion

Low-code is not inherently unsuitable for enterprise use; suitability depends on the workload, platform limits, security needs, and team capability. The mistake is treating rapid initial development as proof that the platform can support every future requirement.

Evaluate using a prototype, realistic data volumes, latency and availability tests, integration failure tests, and a total-cost analysis that includes licensing and maintenance.

Conclusion: use low-code where visual development creates genuine leverage, and use pro-code services where complexity, performance, or control demands it. A hybrid architecture often provides a more defensible balance than either extreme.

Low-code suitability should be assessed against explicit requirements rather than broad claims that the platform is either easy or limited. Forms and approvals may be a strong fit, while a high-frequency optimization engine or a strict real-time service may need a different runtime. The decisive question is whether the complete workload can meet measurable requirements safely and maintainably.

A hybrid architecture can preserve the speed of low-code for user-facing workflows while isolating complex computation in a tested service. The boundary should be explicit: the low-code application collects and displays information, while a service performs specialized scheduling or optimization and returns a versioned response. This prevents complex algorithms from becoming scattered across visual workflows.

Before selecting a platform, test the riskiest requirement with realistic data. Measure latency, throughput, recovery from integration failures, and the effort needed to change business rules. Include licensing, training, migration, vendor dependence, and long-term maintenance in the cost comparison rather than comparing only initial build speed.

The case study supports a selective adoption strategy: begin with low-risk workflows, validate integration boundaries, and move higher-criticality functions only when performance, reliability, and governance have been demonstrated. Architecture decisions should be justified with evidence.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

Learning outcomes

- Compared low-code and pro-code trade-offs.
- Designed a hybrid architecture.
- Connected technical choice to enterprise risk.

Conclusion

Conclusion: use low-code where visual development creates genuine leverage, and use pro-code services where complexity, performance, or control demands it. A hybrid architecture often provides a more defensible balance than either extreme.

References

Zoho Creator: https://www.zoho.com/creator/

NIST SP 800-218 Secure Software Development Framework: https://csrc.nist.gov/pubs/sp/800/218/final

NIST Privacy Framework: https://www.nist.gov/privacy-framework

6. Decision Framework for Low-Code Versus Pro-Code

Classify requirements by criticality, complexity, volume, latency, integration count, audit requirements, and change frequency. Low-code is attractive when forms and workflows dominate; pro-code becomes more attractive when algorithms, concurrency, automated testing, or specialized runtime control dominate.

Prototype the riskiest requirement early. If the enterprise use case requires a strict response-time guarantee, test it under realistic load before choosing the platform. If the hardest requirement is a complex scheduling solver, validate that solver separately rather than building the entire interface first.

Consider total cost of ownership: initial build speed, licensing, integration work, maintenance, staff skills, migration risk, and vendor lock-in. The cheapest first prototype may not be the least expensive system to operate for five years.

Platform limits should be validated using a proof of concept rather than inferred from marketing language. A test should use representative concurrent users, realistic record sizes, and the same integrations planned for the final system. Record both successful response time and behaviour during failure.

Migration and vendor dependence also matter. If a workflow is implemented in platform-specific scripts, moving it elsewhere may require rewriting logic and rebuilding permissions. Keep business rules documented and separate core algorithms from UI configuration when portability is important.

Practical application to the lab

The Gym Enrollment app is a small, well-bounded workflow and is suitable for a low-code tool. To study platform limits, this report considers a hypothetical enterprise case: a multi-region hospital resource orchestration system that coordinates beds, staff, operating theatres, diagnostics, and emergency escalations. This is a case study, not a claim that the gym app already implements hospital operations.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24301012

7. Risk Register and Final Recommendations

High-risk items for the hospital case include incorrect resource allocation, stale data, integration outages, unauthorized access, and a solver that fails under peak load. Each risk needs an owner, detection method, mitigation, and recovery path.

Use staged adoption: pilot low-risk administrative tasks first, then integrate higher-impact workflows only after reliability, auditability, and safety requirements are demonstrated.

Final recommendation: choose the architecture from measured requirements, not from the appeal of a fast prototype or the assumption that low-code cannot scale.

Platform limits should be validated using a proof of concept rather than inferred from marketing language. A test should use representative concurrent users, realistic record sizes, and the same integrations planned for the final system. Record both successful response time and behaviour during failure.

Migration and vendor dependence also matter. If a workflow is implemented in platform-specific scripts, moving it elsewhere may require rewriting logic and rebuilding permissions. Keep business rules documented and separate core algorithms from UI configuration when portability is important.

Practical application to the lab

The Gym Enrollment app is a small, well-bounded workflow and is suitable for a low-code tool. To study platform limits, this report considers a hypothetical enterprise case: a multi-region hospital resource orchestration system that coordinates beds, staff, operating theatres, diagnostics, and emergency escalations. This is a case study, not a claim that the gym app already implements hospital operations.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

2414012012